Maintainer Handbook

Thank you for volunteering as a maintainer for this event.

This event is designed to introduce students to real-world open source contribution culture in a beginner-friendly environment through practical GitHub collaboration.

Participants will contribute to projects through:

- GitHub issues
- branching workflows
- pull requests
- code reviews
- collaborative development practices

The event is inspired by programs such as:

- Hacktoberfest
- GSSoC
- SWOC
- open source community contribution programs

However, the focus here is not just coding — it is about learning:

- collaboration
- GitHub workflows
- reading existing codebases
- contributing to ongoing projects
- professional software development practices

Maintainers are one of the most important parts of this ecosystem.

A contributor’s experience depends heavily on:

- issue quality
- repository clarity
- PR review quality
- maintainer responsiveness
- communication style

This handbook defines the standards and expectations for all maintainers.

Event Vision & Goals

The event aims to:

- introduce students to open source contribution culture
- reduce fear/intimidation around GitHub contributions
- help beginners learn practical workflows
- create collaborative coding culture within campus
- prepare students for external OSS programs
- encourage long-term contribution habits

The event is NOT intended to:

- encourage spam contributions
- prioritize quantity over quality
- become a PR farming contest
- reward meaningless edits

The emphasis should always remain on meaningful learning and collaboration.

Maintainer Role Overview

Maintainers are responsible for:

- preparing repositories
- creating contribution opportunities
- managing issues
- reviewing PRs
- guiding contributors
- maintaining code quality
- ensuring smooth contributor experience

Maintainers act as:

- project owners
- reviewers
- mentors
- facilitators

Maintainers are expected to maintain professionalism and consistency throughout the event.

Maintainer Expectations

Maintainers are expected to:

Before the Event

- prepare the project repository
- ensure setup works correctly
- add documentation
- create issues
- organize labels
- test contribution workflow

During the Event

- actively review PRs
- answer contributor questions
- assign/manage issues
- maintain contribution quality
- prevent spam/low-effort contributions
- merge valid PRs
- ensure contributors follow guidelines

General Expectations

- remain reasonably active
- communicate professionally
- guide beginners patiently
- avoid favoritism or bias
- maintain fairness in reviews

1. Repository Preparation

Projects should be contribution-ready before the event starts.

1.1 Project Quality

Projects do NOT need to be production-grade.

However, projects MUST:

- run correctly
- have understandable structure
- contain meaningful scope for contribution
- support collaborative development

Avoid:

- unfinished broken repositories
- random experimental code
- poorly structured codebases
- projects without setup instructions

1.2 Recommended Repository Structure

Repositories should ideally contain:

- README.md
- CONTRIBUTING.md
- LICENSE
- issue templates
- PR template
- screenshots/assets
- environment setup instructions

2. Documentation Standards

Documentation quality directly impacts contributor experience.

2.1 README.md

The README should clearly explain:

Mandatory Sections

- project overview
- features
- tech stack
- screenshots/demo
- setup instructions
- installation steps
- run commands
- contribution instructions

Setup Instructions Must Include

- prerequisites
- dependencies
- environment variables
- database setup (if needed)
- API keys (if applicable)
- commands to start project

A contributor should be able to set up the project without needing external help.

2.2 CONTRIBUTING.md

Should include:

- contribution workflow
- branch naming format
- commit naming format

- PR process
- issue claiming process
- coding conventions
- review expectations

2.3 PR Template

Recommended sections:

- linked issue
- changes made
- screenshots
- testing performed
- checklist

---

3. Issue Creation Guidelines

Issue quality determines contributor experience.

Poorly written issues create:

- confusion
- duplicate work
- unnecessary doubts
- low-quality submissions

3.1 Every Issue Should Include

Title

Clear and specific.

GOOD:
✅ “Add dark mode toggle to navbar”

BAD:
❌ “Improve app”

Description

Mention:

- current problem
- expected behavior
- relevant files/components
- screenshots if needed
- acceptance criteria

Labels

Use:

- difficulty labels
- category labels

Difficulty

Specify:

- beginner
- easy
- medium
- hard

3.2 Beginner-Friendly Issues

Every project MUST contain beginner-friendly issues.

Recommended: At least 40–50% beginner/easy issues.

Examples:

- responsiveness fixes
- text changes
- UI cleanup
- README improvements
- small bug fixes
- loading states

3.3 Advanced Issues

Hard issues should:

- have clear expectations
- avoid ambiguity
- not block contributors unnecessarily

---

4. Difficulty Classification

Beginner

Requires minimal project understanding.

Examples:

- CSS fixes
- text corrections
- documentation
- responsiveness

Easy

Requires basic understanding.

Examples:

- validation
- small features
- reusable components

Medium

Requires project understanding.

Examples:

- API integration
- auth systems

- database operations

Hard

Requires architecture understanding.

Examples:

- optimization
- refactoring
- advanced backend systems

---

5. Labels & Repository Organization

Recommended labels:

Difficulty

- beginner
- easy
- medium
- hard

Categories

- frontend
- backend
- fullstack
- documentation
- bug
- enhancement
- UI/UX
- testing

Contribution Labels

- good first issue
- help wanted

Consistent labeling improves discoverability.

6. Contribution Workflow

Recommended workflow:

1. Contributor explores issues
2. Contributor requests assignment
3. Maintainer assigns issue
4. Contributor creates branch
5. Contributor works on issue
6. Contributor raises PR
7. Maintainer reviews PR
8. Contributor makes requested changes
9. PR merged if approved

---

7. Issue Assignment Rules

Recommended rules:

- one active issue per contributor initially
- inactivity timeout (24–48 hours)
- issue reassignment allowed if inactive
- no duplicate assignments
- contributors should comment before working

---

8. Pull Request Workflow

Contributors should:

- create clean PRs
- link issue properly
- explain changes clearly
- avoid unrelated modifications

Maintainers should:

- review carefully
- provide feedback
- request improvements if needed

9. PR Review Standards

9.1 Review Time

Target review time: within 12 hours whenever possible.

Delayed reviews reduce contributor motivation.

9.2 Review Behaviour

Maintainers should:

- remain respectful
- explain mistakes clearly
- encourage learning
- avoid harsh criticism

9.3 Good Review Examples

✅
“Good implementation. Please make the component responsive for smaller screens as well.”

✅
“Functionality works correctly. Please improve variable naming for readability.”

9.4 Poor Review Examples

❌ “Wrong.”

❌ “Bad code.”

❌ “Do again.”

10. Contributor Interaction Guidelines

Many contributors may be:

- beginners
- first-time GitHub users
- unfamiliar with OSS workflow

Maintainers should:

- answer doubts patiently
- encourage contributors
- guide instead of dismissing
- maintain welcoming environment

The goal is learning, not intimidation.

---

11. Spam & Low-Quality Contribution Handling

Maintainers should reject:

- meaningless PRs
- unnecessary changes
- duplicate contributions
- copied solutions
- spam commits
- unrelated modifications

Examples:

- changing random colors repeatedly
- unnecessary README edits
- formatting-only spam
- tiny meaningless changes for points

---

12. AI Usage Policy

AI-assisted coding may be allowed depending on event policy.

However:

- contributors must understand submitted code
- maintainers may ask contributors to explain implementations
- blindly generated code should not be accepted

Maintainers should prioritize:

- correctness
- understanding
- maintainability

---

13. Code Quality Standards

Maintainers should ensure:

- code readability
- consistency
- proper formatting
- maintainability
- no breaking changes

Avoid merging:

- untested code
- poorly structured implementations
- large unrelated changes

---

14. Activity & Response Expectations

Maintainers are expected to remain active during the event.

Recommended:

- check repos regularly
- respond to doubts quickly
- review PRs consistently
- monitor discussions

If unavailable:

- inform organizing team early
- coordinate with co-maintainers

15. Scoring & Validation

If scoring exists:

- only meaningful merged PRs should count
- spam contributions should not be rewarded
- maintainers may flag suspicious contributions

Maintainers should not:

- unfairly inflate points
- favor friends
- merge low-quality PRs

---

16. Conflict & Dispute Handling

If disputes occur:

- remain professional
- avoid public arguments
- escalate major issues to organizing team

Examples:

- duplicate work disputes
- plagiarism accusations
- unfair scoring complaints

---

17. Event Communication

Maintainers should:

- monitor communication channels
- announce major updates
- clarify doubts publicly when possible
- avoid inconsistent instructions

18. Maintainer Checklist

Before Event

✅ Repo setup complete
✅ Setup tested
✅ README added
✅ CONTRIBUTING added
✅ Labels created
✅ Issues created
✅ Beginner issues added
✅ PR template added

During Event

✅ Assign issues
✅ Review PRs actively
✅ Guide contributors
✅ Prevent spam
✅ Merge valid PRs
✅ Maintain professionalism

---

19. Common Mistakes To Avoid

Avoid:

- vague issues
- delayed reviews
- ignoring contributors
- merging low-quality PRs
- creating only hard issues
- poor documentation
- inconsistent rules
- rude communication